Prática
Abrindo o capô de um modelo 3D gerado por IA

Pedro Ximenes

Universidade Católica de Pernambuco (UNICAP)

5 de novembro de 2026

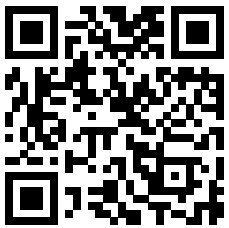
Como vai funcionar

- Em **duplas**, com **um computador** e **uma ficha** por dupla.
- Tudo roda no **navegador**: nada para instalar e nenhuma conta obrigatória.
- Respondam a ficha **enquanto fazem**. Não deixem para o fim.

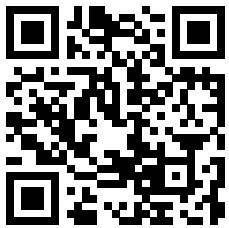
1. Modelo	2. Abrir o capô (Three.js Editor)	3. Splat	Ficha



Passo 1 (opcional):
Hunyuan3D



Passo 2:
`threejs.org/editor`

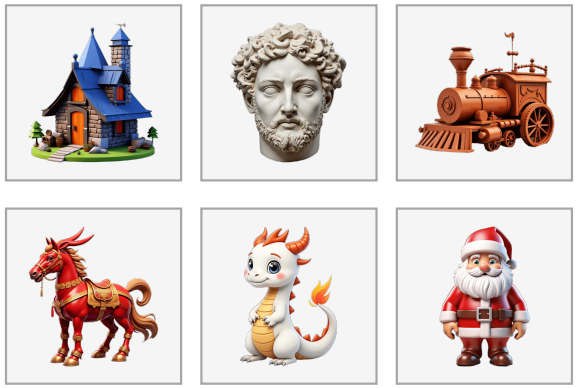


Passo 3:
`antimatter15.com/splat`

Passo 1 – Pegar (ou gerar) um modelo 3D

Demonstração: o professor gera um modelo ao vivo.

Cada dupla escolhe um modelo .glb da **pasta da turma**. Todos foram gerados por IA a partir das imagens ao lado. PC lento? Usem a versão `_leve`.



imagens de entrada usadas pela IA

(pasta `imagens_entrada/`)

Querem gerar o de vocês? (opcional)

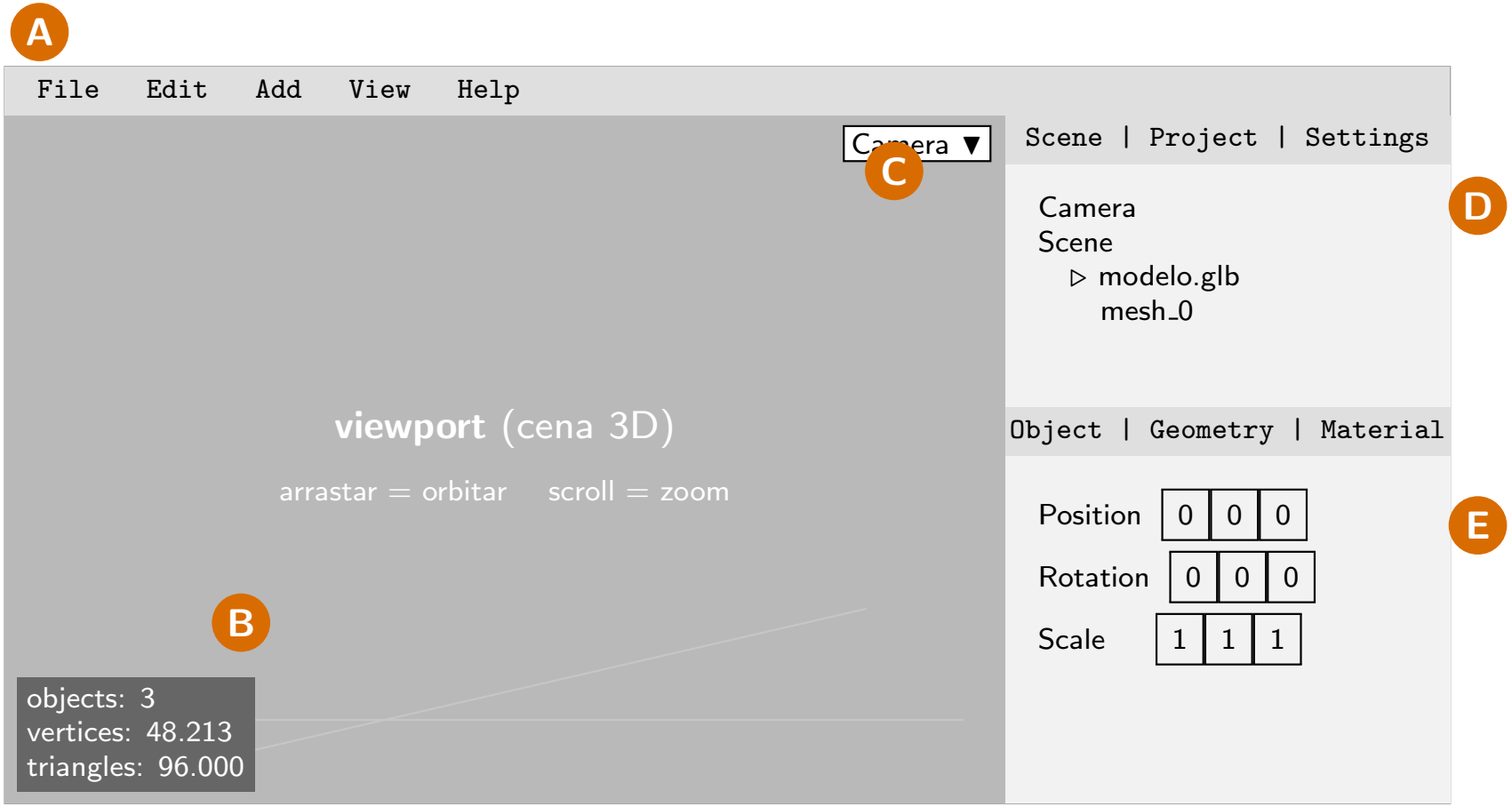
Com conta **gratuita** no Hugging Face (logados): `huggingface.co/spaces/tencent/Hunyuan3D-2` enviem uma imagem com **fundo liso** → `Gen Shape` → baixem o **.glb**.

Sem login, a cota de GPU acaba em 1–2 gerações para o laboratório **inteiro**.

Ficha – questão 1

Qual modelo vocês escolheram? Olhando só a imagem de entrada, o que acham que a IA vai errar?

Passo 2 – O Three.js Editor



A menu: `File` → `Import`
B contadores: vértices e triângulos
C qual câmera o viewport usa
D lista de objetos da cena (*outliner*)
E propriedades do objeto selecionado

2a – Importar e olhar a malha

1. Abram `threejs.org/editor` e usem `File` → `Import` para abrir o **.glb**.
   (O modelo pode aparecer pequeno ou grande demais. Usem o scroll para dar zoom.)
2. Anotem os contadores do canto inferior esquerdo (**B**): **vértices** e **triângulos**.
3. Na lista de objetos (**D**), **expandam** o modelo e cliquem na **malha** (*mesh*) de dentro.
4. Aba `Material` → marquem `Wireframe`.

Lembrem

Todo modelo 3D é feito de **vértices** ligados em **triângulos**, as primitivas que a rasterização transforma em pixels.

Ficha – questão 2

Quantos triângulos? A malha parece feita por uma pessoa ou é "bagunçada"?

2b – Transformações geométricas

Selecionem o **modelo** e, na aba `Object` (**E**), digitem valores em **Position**, **Rotation** (graus) e **Scale**.

Atalhos no viewport: [W] transladar [E] rotacionar [R] escalar (arrastem as setas/anéis).

Tentem:

- Scale `(2, 2, 2)`: fica o dobro.
- Scale `(1, -1, 1)`: o que acontece?
- Rotation `(0, 90, 0)`: gira em torno de qual eixo?
- Position `(3, 0, 0)` e depois Rotation `(0, 45, 0)`: girou "no lugar" ou em volta da origem?

Ordem das transformações

Não importa a ordem em que vocês digitam: o Three.js sempre monta

$$M = T \cdot R \cdot S$$

ou seja, aplica **primeiro a escala**, depois a rotação, depois a translação.

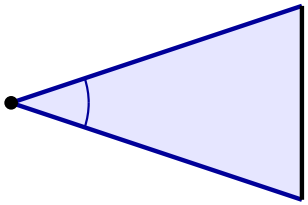
Ficha – questão 3

Anotem os valores que usaram e expliquem: por que o objeto girou "no lugar" mesmo depois de transladado?

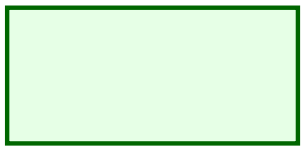
2c – Câmera e projeção

FOV (abertura da câmera perspectiva)

1. Na lista (**D**), cliquem em **Camera**.
2. Na aba `Object`, mudem o **FOV**: 50 → 20 → 90.



perspectiva (FOV)



ortogonal (volume = caixa)

Projeção ortogonal

1. `Add` → `Camera` → `Orthographic`
   (em versões antigas: `Add` → `OrthographicCamera`)
2. Com ela selecionada, na aba `Object`: **Position** `(0, 0, 5)`.
   (ela nasce na origem, ou seja, dentro do modelo!)
3. No seletor do viewport (**C**), escolham a câmera ortográfica.
4. Mudem **Left/Right/Top/Bottom** (ex.: `-2, 2, 2, -2`): é a **janela de visão**!

Ficha – questão 4

O que acontece com FOV 20 e 90? Que diferença visível há entre perspectiva e ortográfica?

2d – Caçando os erros da IA

Voltem para a câmera perspectiva, desmarquem o wireframe e **girem o modelo 360°**.

Procurem:

- ☐ **Costas inventadas**: o lado que a foto não mostrava.
- ☐ **Buracos** ou partes "grudadas".
- ☐ Detalhes finos que **sumiram** (dedos, alças, textos).
- ☐ **Triângulos demais** em áreas planas.
- ☐ Base torta ou objeto "afundando".



a IA só viu **este** lado da casa

Ficha – questão 5

Que erros vocês encontraram? Um artista 3D poderia usar esse modelo direto num jogo?

Passo 3 – Um Gaussian Splat no navegador

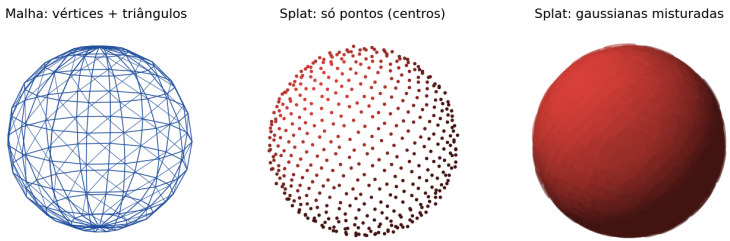
1. Abram `antimatter15.com/splat`
   (a cena carrega sozinha; pode levar alguns segundos)
2. Naveguem:

arrastar o mouse	orbitar
scroll	zoom
W A S D / setas	andar
0 . . . 9	câmeras das fotos originais

3. Cheguem **bem perto** de um objeto.
   O que aparece?

Lembrem

Não há triângulos: são milhões de gaussianas **projetadas** e **misturadas** na tela.



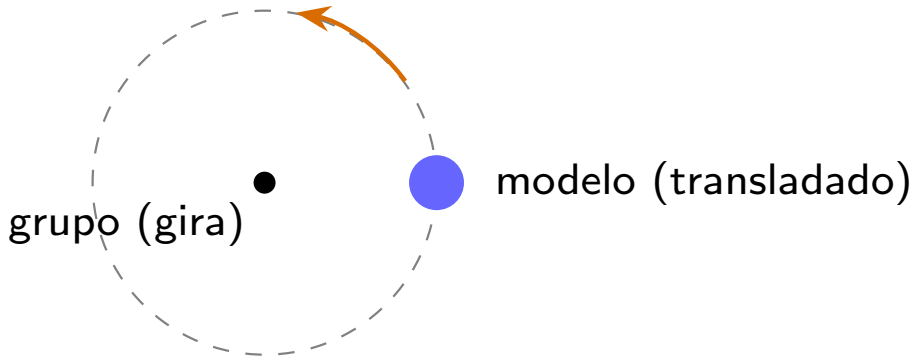
Ficha – questão 6

De perto, como o splat se parece? Que diferença vocês veem em relação à malha do passo 2? Quando usar cada um?

Desafio extra (se sobrar tempo)

Hierarquia de transformações, como no Cubo Mágico e no Sistema Solar:

1. `Add` → `Group`.
2. Na lista (**D**), **arrastem o modelo para dentro** do grupo.
3. Deem **Position** `(3, 0, 0)` ao **modelo** e **Rotation** ao **grupo**.
4. O modelo agora **orbita** a origem. Por quê?



$$M_{\text{final}} = M_{\text{grupo}} \cdot M_{\text{modelo}} = R_{\text{grupo}} \cdot T_{\text{modelo}}$$

Fechamento – discussão rápida

1. A IA gerou o modelo, mas **o que vocês usaram do semestre** para examinar e mexer nele?
2. Vocês colocariam esse modelo direto num jogo? O que precisaria corrigir?
3. Malha ou splat: qual vocês escolheriam para...
   - um personagem que anda e pula?
   - um tour virtual de um museu?

Entrega

Entreguem a ficha com o nome da dupla. Obrigado pelo semestre!